

Fundamentos de Spring Boot

1. Fundamentos de Spring Boot

Fundamentos de Spring Boot

Hasta ahora todo lo que has programado en Java ha sido consola: el programa arranca, hace algo, y termina. A partir de aquí vas a construir un tipo de programa distinto — uno que arranca y se queda esperando, sin que tú le digas nada más, hasta que otro programa le hace una pregunta.

Este apartado es la base sin la que no vas a entender nada de lo que viene, ni en Acceso a Datos ni en Programación de Servicios y Procesos (PSP).

De la consola al servidor

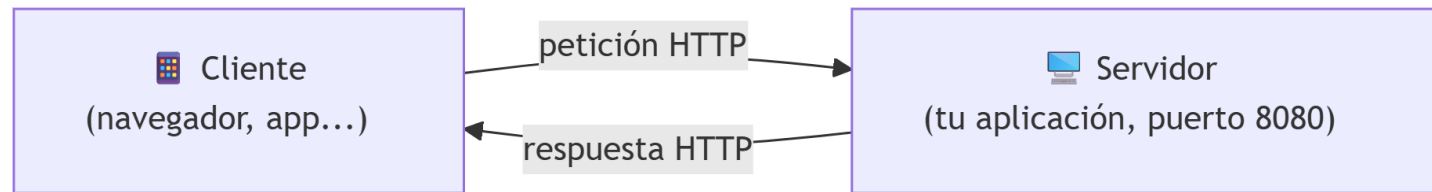
Un programa de consola tiene un ciclo de vida corto: arranca, hace su trabajo, y termina. Una aplicación de servidor (backend) es distinta: arranca, se pone a escuchar en un puerto, y se queda ahí, viva, esperando. Su "usuario" es otro programa (un navegador, una app, otro servidor) que le habla por la red.

- Dirección IP — identifica la máquina dentro de la red (qué ordenador es).
- Puerto — identifica, dentro de esa máquina, a qué programa concreto va dirigido el mensaje.

Analogía del bloque de pisos: la IP es la dirección del edificio; el puerto es el número de piso. Sin el número de piso, el cartero llega al portal pero no sabe dónde dejar la carta. Un servidor "escuchando en el puerto 8080" espera mensajes dirigidos exactamente a ese número, en esa máquina.

De la consola al servidor (cont.)

Cliente y servidor necesitan hablar el mismo idioma: un formato de mensaje acordado de antemano, llamado protocolo. El que vas a usar todo el curso es HTTP — el mismo que usa tu navegador cada vez que visitas una página.



El servidor no decide cuándo actuar, el cliente sí: en un programa de consola tú controlas el ritmo; un servidor se queda a la espera y reacciona cada vez que llega una petición.

La aplicación que vas a construir es exactamente eso: un programa Java sin Scanner ni System.out.println esperando Intro — arranca, escucha en un puerto, y responde peticiones HTTP hasta que alguien lo detiene.

Qué es un framework

Hasta ahora, en tus programas, el flujo de control lo llevas tú: tu `main()` llama a tus clases, tus clases llaman a librerías cuando las necesitan. Un framework invierte esa relación: es la librería la que llama a tu código, cuando ella decide (inversión de control).

Analogía del restaurante: cocinar en casa es tu programa de consola — decides tú el orden. Trabajar con un framework es ser cocinero de restaurante: el camarero (el framework) avisa "mesa 4, primer plato" y tú escribes solo la receta. Cuándo se sirve y en qué orden lo decide el camarero.

Spring es el framework de Java más usado para este tipo de aplicaciones. Spring Boot añade sobre Spring: servidor web ya integrado, configuración automática, y starters — paquetes que agrupan de golpe todas las dependencias que necesitas para una funcionalidad.

Maven y el pom.xml

Un proyecto Java real depende de librerías externas (Spring, el driver de PostgreSQL, Lombok...). Maven gestiona esas dependencias por ti: declaras qué necesitas en el pom.xml y Maven descarga cada .jar con la versión correcta, resolviendo también sus propias dependencias.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

groupId identifica quién publica la librería; artifactId identifica cuál de sus paquetes quieres. Con esa entrada, Maven descarga ese starter y todo lo que necesita para funcionar.

Maven y el pom.xml (cont.)

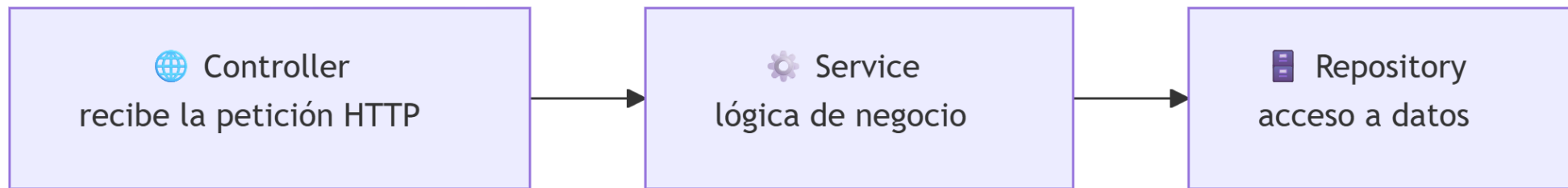
Las dos dependencias clave de este curso:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

El nombre del starter web ha cambiado: en Spring Boot 4 se separó spring-boot-starter-web en spring-boot-starter-webmvc (el que vas a usar) y spring-boot-starter-webflux (reactivo, fuera de este curso). Si buscas spring-boot-starter-web a secas en tu pom.xml, no lo vas a encontrar.

La arquitectura en capas

Una aplicación backend reparte el código en capas, cada una con una responsabilidad clara. El patrón más extendido — y el que vas a usar todo el curso — tiene tres:



Ejemplo — una aplicación que gestiona el catálogo de una librería (LibroController / LibroService / LibroRepository): el controller enruta, el service decide, el repository habla con la base de datos.

La arquitectura en capas (cont.)

- Controller (LibroController) — recibe la petición HTTP y la traduce a una llamada Java. No contiene lógica de negocio, solo enruta.
- Service (LibroService) — contiene la lógica: qué hay que comprobar, qué hay que calcular, en qué orden.
- Repository (LibroRepository) — es quien habla con la base de datos.

¿Por qué separar así? Cada capa se puede entender, probar y cambiar sin tocar las demás: si cambia la base de datos, solo se toca el repository; si cambia una regla de negocio, solo el service. Regla: cada capa habla solo con la siguiente — el controller nunca accede directamente a la base de datos.

Inyección de dependencias

El controller necesita un service, y el service necesita un repository. ¿Quién crea esos objetos y los encaja unos dentro de otros — tu código, o el framework? La forma más directa es crearlos tú mismo:

```
public class LibroService {  
    private LibroRepository repositorio = new LibroRepository();  
}
```

Problema: el service queda unido a esa implementación concreta. Si durante un test quieres sustituir el repositorio real por uno simulado, no puedes — el `new LibroRepository()` está escrito dentro de la propia clase.

Inyección de dependencias (cont.)

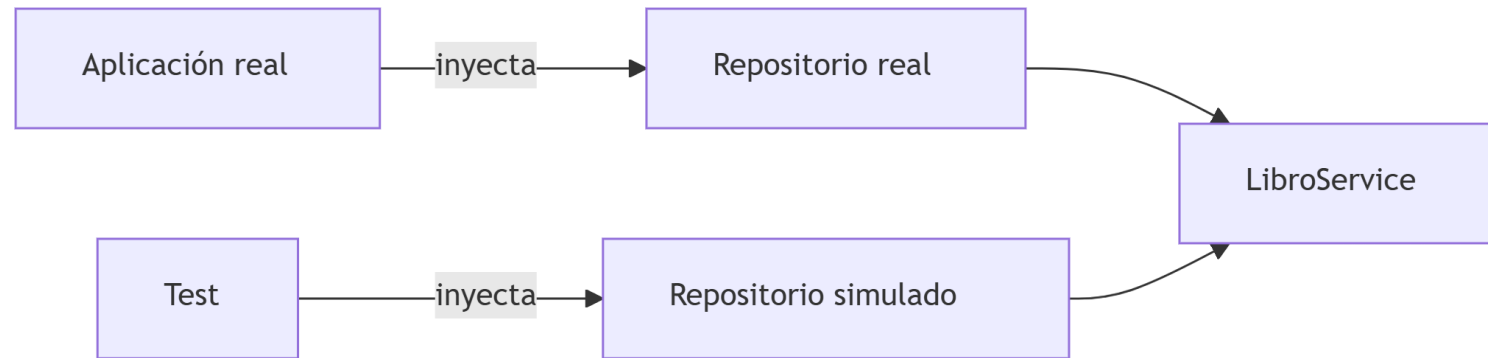
Con inyección de dependencias (DI), el service deja de construir el objeto que necesita: lo declara como parámetro del constructor.

```
public class LibroService {  
  
    private final LibroRepository repositorio;  
  
    public LibroService(LibroRepository repositorio) {  
        this.repositorio = repositorio;  
    }  
}
```

LibroService sigue usando el repositorio igual, pero ya no decide cómo se crea ni qué implementación concreta recibe. Solo declara: «para funcionar necesito un LibroRepository».

Inyección de dependencias (cont.)

Esto permite cambiar la dependencia sin modificar el código del service: en la aplicación normal recibe el repositorio real; durante un test, uno simulado.



La inyección de dependencias consiste en que una clase reciba desde fuera los objetos que necesita, en vez de construirlos ella misma. Es una aplicación concreta de la inversión de control.

Inyección de dependencias (cont.)

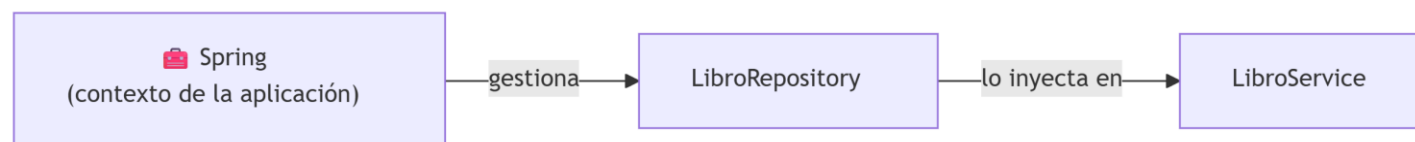
¿Quién construye el LibroRepository y decide dárselo al LibroService? La respuesta es Spring. Al arrancar, Spring crea y guarda en un contenedor interno — el contexto de la aplicación — una instancia de cada clase gestionada: esas instancias son los beans.

La misma analogía del restaurante, un paso más: no vas tú al almacén a buscar los ingredientes — te los traen a la mesa de trabajo, ya listos. LibroService no va a buscar su LibroRepository: se lo entregan.

Sin inyección de dependencias:



Con inyección de dependencias:



Inyección de dependencias (cont.)

Para que Spring sepa qué clases debe gestionar como beans, se marcan con anotaciones:

Anotación	Qué declara
@Service	Un bean con lógica de negocio (capa intermedia entre controlador y datos).
@Repository	Un bean que accede a datos (normalmente, una interfaz que Spring Data implementa por ti).
@RestController	Un bean que expone endpoints HTTP y devuelve las respuestas en el cuerpo (JSON).
@Configuration	Una clase que define manualmente uno o varios beans, para casos que no encajan arriba.

Las cuatro tienen algo en común: todas le dicen a Spring "esto es un bean, gestióname el ciclo de vida y pásame lo que necesite por inyección".

Inyección de dependencias (cont.)

El mecanismo se repite hacia arriba. Así se ve la cadena completa en el controller de la librería:

```
@RestController
@RequestMapping("/api/v1/libros")
public class LibroController {
    private final LibroService libroService;
    public LibroController(LibroService libroService) {
        this.libroService = libroService;
    }
    @GetMapping("/{id}")
    public ResponseEntity<LibroDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

No hace falta entender todavía `@GetMapping`, `@PathVariable` ni `ResponseEntity` — son de HTTP/REST y las verás en detalle en PSP. Aquí solo interesa la forma: controller → service → repository, cada uno con su responsabilidad, sin saltarse capas.

El punto de arranque de la aplicación

Toda aplicación Spring Boot tiene una clase con un main() que la arranca:

```
@SpringBootApplication
public class LibreriaApplication {

    static void main(String[] args) {
        SpringApplication.run(LibreriaApplication.class, args);
    }
}
```

@SpringBootApplication le dice a Spring Boot "escanea este paquete y los que cuelgan de él buscando beans, y configúrate automáticamente según lo que encuentres en el classpath".
SpringApplication.run(...) arranca de verdad el servidor embebido y deja la aplicación escuchando.

El @RequiredArgsConstructor de Lombok

Escribir a mano el constructor de inyección de cada clase se vuelve repetitivo enseguida. Lombok genera código repetitivo por ti a partir de anotaciones; @RequiredArgsConstructor genera automáticamente un constructor con un parámetro por cada campo final de la clase:

```
@Service
@RequiredArgsConstructor
public class LibroService {
    private final LibroRepository libroRepository;
    private final EditorialRepository editorialRepository;
    // Lombok genera aquí, en tiempo de compilación,
    // un constructor con ambos repositorios.
}
```

Sin Lombok tendrías que escribir ese constructor a mano cada vez que añadieras un campo final. Con la anotación, Spring sigue inyectando exactamente igual (por constructor), pero tú no mantienes ese código repetitivo.

Configuración: `application.yml` y perfiles

Spring Boot centraliza la configuración (credenciales de base de datos, puertos, propiedades propias) en ficheros YAML en `src/main/resources`. Lo habitual es tener al menos dos: `application.yml`, con la configuración común, y `application-dev.yml`, con la del entorno de desarrollo — por ejemplo, la URL de conexión a PostgreSQL de la Actividad 1.1.

`".yml"` o `".yaml"`: Spring Boot reconoce las dos extensiones sin distinción — es convención de quien escribe el fichero, no una regla técnica. En este curso se usa siempre `.yml`, por uniformidad.

Ese `-dev` en el nombre es un perfil: un conjunto de configuración que solo se activa si le dices a Spring Boot que lo use al arrancar. El mismo código puede arrancar con configuración distinta según el entorno (desarrollo, tests, producción) sin tocar una línea de Java.

Dónde estás y hacia dónde vas

Estas piezas —el proyecto Maven, las capas, la inyección de dependencias, la configuración— las vas a trabajar durante todo el curso desde dos módulos a la vez: en Acceso a Datos te centrarás en la persistencia, todo lo que hay del service hacia abajo (JDBC, JPA, JSONB, MongoDB); en Programación de Servicios y Procesos te centrarás en los servicios en red y la seguridad, del controller hacia fuera (REST, JWT, hilos, sockets).

En las actividades de ambos módulos irás construyendo, pieza a pieza, tu propia aplicación con esta misma estructura.

Ideas clave

- Una aplicación de servidor arranca y se queda escuchando en un puerto. La IP identifica la máquina, el puerto el programa dentro de ella, y HTTP es el protocolo con el que cliente y servidor se entienden.
- Un framework invierte el control: en vez de que tu código llame a la librería, la librería llama al tuyo (inversión de control).
- Spring es el framework; Spring Boot añade configuración automática, servidor embebido y starters.
- Maven gestiona las dependencias vía pom.xml: spring-boot-starter-webmvc y spring-boot-starter-data-jpa son las claves de este curso.

Ideas clave (cont.)

- Arquitectura en capas: controller (HTTP) → service (lógica) → repository (datos). Cada capa habla solo con la siguiente.
- Inyección de dependencias: una clase declara lo que necesita por constructor; Spring guarda un bean de cada clase gestionada en su contexto y se lo entrega ya construido. `@Service`, `@Repository`, `@RestController` y `@Configuration` marcan qué gestiona Spring.
- `application.yml` / `application-dev.yml` centralizan la configuración; un perfil permite variarla según el entorno sin tocar código.
- `@RequiredArgsConstructor` de Lombok genera el constructor con los campos final, el que usa Spring para inyectar.